

Data Model Draft

☰ Area	Database
🕒 Created	August 4, 2026 4:42 AM
🕒 Last edited	August 7, 2026 6:39 PM
🕒 Note Type	Research
🕒 Phase	Phase 0 — Discovery & Requirements
🌟 Status	In progress
📄 Sub-Page	Requirements & User Stories

- ☐ Identify the main "things" (entities) the app needs to store. At minimum:
 - ☐ **Products** — name, price, description, images, category, sizes available, stock quantity
 - ☐ **Categories** — name, slug
 - ☐ **Orders** — customer info, items ordered, total price, status, date
 - ☐ **Users** (if implementing accounts) — name, email, hashed password
- ☐ For each entity, list the attributes (fields) you know it needs
- ☐ Draw a rough sketch (even on paper) of how these entities relate to each other (e.g., an Order contains many Products)
- ☐ Save or photograph this sketch and attach it to the  Database Schema Design sub-page as a starting reference



Relational Database

I will be using a relational database to better organize my data and enforce structure and consistency using fixed schemas and relationships

Why?

- Fixed schema is more reliable for stable data shape
- stronger data integrity essential for money-related data
- ACID compliant, preventing partial payment states
- Easier relationship linking tables
- PostgreSQL is always fully ACID compliant (instead of using MySQL with InnoDB engine)
- Complex queries are stronger with PostgreSQL

User Table

Column	Type	Constraints
id	INTEGER GENERATED ALWAYS AS IDENTITY	PRIMARY KEY
first_name	VARCHAR(50)	NOT NULL
last_name	VARCHAR(50)	NOT NULL
email_address	VARCHAR(254)	UNIQUE NOT NULL
password_hash	VARCHAR(60)	NOT NULL
role	VARCHAR(20)	NOT NULL DEFAULT 'customer' CHECK (role IN ('customer', 'admin'))
email_verified	BOOLEAN	NOT NULL DEFAULT FALSE
status	VARCHAR(20)	NOT NULL DEFAULT 'active' CHECK (status IN ('active', 'suspended', 'deleted'))
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()

Design Notes:

- `id`, `first_name`, `last_name`, `email_address`, `password_hash`, `role` cover identity and auth basics
- `created_at`, `updated_at` were added for audit/tracking purposes
 - `created_at` is needed for analytics, debugging and account age
 - `updated_at` is critical for knowing if data is stale and sync/debugging
- `email_verified` tracks whether user confirmed their email, essential for ensuring customer receives their order confirmation and tracking updates
- `status` enables disabling of accounts without permanently deleting the order history tied to account
- `role` restricts accounts to a known set of permission levels, enabling role-based access control between customer and admin users
- `password_hash` to avoid storing plaintext

Address Table

Column	Type	Constraints
id	INTEGER GENERATED ALWAYS AS IDENTITY	PRIMARY KEY
address_line_1	VARCHAR(100)	NOT NULL
address_line_2	VARCHAR(100)	
city	VARCHAR(85)	NOT NULL
region	VARCHAR(50)	
country	VARCHAR(56)	NOT NULL
postal_code	VARCHAR(20)	
address_type	VARCHAR(20)	NOT NULL CHECK (address_type IN ('shipping', 'billing'))
is_default	BOOLEAN	NOT NULL DEFAULT FALSE
recipient_name	VARCHAR(100)	NOT NULL

Column	Type	Constraints
phone_number	VARCHAR(20)	
user_id	INTEGER	NOT NULL REFERENCES users(id)
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()

Design Notes

- `region` - set to `nullable` as not all countries use a state/province-level division
- `country` - stores as ISO to standardize country names and avoid inconsistency
- `postal_code` - also set to `varchar nullable` as some countries do not use postal codes and some countries postal codes contain letters
- `is_default` - allows returning customers to auto select their preferred address rather than making them pick every time
- `recipient_name` - person receiving the package isn't always the account holder
- `phone_number` - phone number tied to delivery address for shipping carriers

Category Table

Column	Type	Constraints
id	INTEGER GENERATED ALWAYS AS IDENTITY	PRIMARY KEY
slug	VARCHAR(100)	UNIQUE NOT NULL
name	VARCHAR(50)	UNIQUE NOT NULL
parent_category_id	INTEGER	REFERENCES categories(id)
description	TEXT	
image_url	TEXT	
is_active	BOOLEAN	NOT NULL DEFAULT TRUE
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()

Design Notes

- `slug` - URL-friendly version of the name for clean page URL
- `parent_category_id` - self-referencing foreign key for category depth
- `description` - stores category page copy
- `image_url` - stores category thumbnail image
- `is_active` - provides more flexibility with hiding categories temporarily without deleting

Product Variant Table

Column	Type	Constraints
id	INTEGER GENERATED ALWAYS AS IDENTITY	PRIMARY KEY
product_id	INTEGER	NOT NULL REFERENCES products(id)
sku	VARCHAR(50)	UNIQUE NOT NULL
size	VARCHAR(20)	NOT NULL DEFAULT 'One Size'
color	VARCHAR(30)	
price_override	DECIMAL(10, 2)	
inventory_id	INTEGER	NOT NULL UNIQUE REFERENCES inventory(id)
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()

Design Notes

- `product_id` - foreign key that links back to the general product
- `sku` - unique code per variant
- `size , color` - stores options for each product
- `inventory_id` - foreign key to inventory for the specific product variant

- `price_override` - only used when a variant's price differs from the base product price

Product Table

Column	Type	Constraints
id	INTEGER GENERATED ALWAYS AS IDENTITY	PRIMARY KEY
category_id	INTEGER	REFERENCES categories(id)
title	VARCHAR(100)	NOT NULL
image_url	TEXT	
description	TEXT	
price	DECIMAL(10,2)	NOT NULL
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()
status	VARCHAR(20)	NOT NULL CHECK (status IN ('active', 'disabled'))

Design Notes

- inventory and discount logic live on product_variants and orders table because stock and pricing promotions apply per-variant/transaction, not the general product

Inventory Table

Column	Type	Constraints
id	INTEGER GENERATED ALWAYS AS IDENTITY	PRIMARY KEY
quantity	INTEGER	NOT NULL DEFAULT 0 CHECK (quantity ≥ 0)

Column	Type	Constraints
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()

Design Notes

- one-to-one with product_variants: each SKU has its own tracked stock count

Discount Table

Column	Type	Constraints
id	INTEGER GENERATED ALWAYS AS IDENTITY	PRIMARY KEY
name	VARCHAR(50)	UNIQUE NOT NULL
description	TEXT	
discount_percent	DECIMAL(5,2)	NOT NULL CHECK (discount_percent > 0 AND discount_percent <= 100)
is_active	BOOLEAN	NOT NULL DEFAULT FALSE
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()
code	VARCHAR(20)	UNIQUE NOT NULL
start_date	TIMESTAMPTZ	NOT NULL
end_date	TIMESTAMPTZ	NOT NULL
usage_limit	INTEGER	CHECK (usage_limit > 0)
usage_count	INTEGER	NOT NULL DEFAULT 0 CHECK (usage_count ≥ 0)

Design Notes

- discount_percent - set to decimal(5, 2) for percentage rate
- `is_active` - allows admin to activate/disable promo as well as query which discount codes are active/not

- `start_date/end_date` - sets a specific time frame the discount code is allowed to be used
- `usage_limit / usage_count` - caps how many times a code can be redeemed as well as track redemptions so far

Order Table

Column	Type	Constraints
id	INTEGER GENERATED ALWAYS AS IDENTITY	PRIMARY KEY
user_id	INTEGER	NOT NULL REFERENCES users(id)
status	VARCHAR(20)	NOT NULL DEFAULT 'pending' CHECK (status IN ('pending', 'paid', 'shipped', 'delivered', 'cancelled'))
total	DECIMAL(10,2)	NOT NULL CHECK(total ≥ 0)
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()
shipping_address_id	INTEGER	NOT NULL REFERENCES addresses(id)
billing_address_id	INTEGER	NOT NULL REFERENCES addresses(id)
discount_id	INTEGER	REFERENCES discounts(id)

Design Notes

- `shipping_address_id` and `billing_address_id` both reference the addresses table but serve distinct purposes
- `discount_id` captures which promo was applied to the transaction if any
- one to many relationship with order_items table, one order can contain many order_items

Order_Items Table

Column	Type	Constraints
id	INTEGER GENERATED ALWAYS AS IDENTITY	PRIMARY KEY
order_id	INTEGER	NOT NULL REFERENCES orders(id)
product_variant_id	INTEGER	NOT NULL REFERENCES product_variants(id)
quantity	INTEGER	NOT NULL CHECK (quantity > 0)
unit_price	DECIMAL(10,2)	NOT NULL CHECK (unit_price > 0)
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()

Design Notes

- unit_price stores a snapshot of the price paid at time of purchase, preserving historical order accuracy

Cart Table

Column	Type	Constraints
id	INTEGER GENERATED ALWAYS AS IDENTITY	PRIMARY KEY
user_id	INTEGER	REFERENCES users(id)
session_id	TEXT	
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()
expires_at	TIMESTAMPTZ	NOT NULL DEFAULT (NOW() + INTERVAL '30 days')

Column	Type	Constraints
		CHECK (expires_at > created_at)

Design Notes

- user_id is nullable and paired with session_id to support guest checkout

Cart Items Table

Column	Type	Constraints
id	INTEGER GENERATED ALWAYS AS IDENTITY	PRIMARY KEY
cart_id	INTEGER	NOT NULL REFERENCES cart(id)
product_variant_id	INTEGER	NOT NULL REFERENCES product_variant(id)
quantity	INTEGER	NOT NULL CHECK (quantity > 0)
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()

Design Notes

Product Images Table

Column	Type	Constraints
id	INTEGER GENERATED ALWAYS AS IDENTITY	PRIMARY KEY
product_id	INTEGER	NOT NULL REFERENCES products(id)

Column	Type	Constraints
image_url	TEXT	NOT NULL
alt_text	VARCHAR(50)	
sort_order	INTEGER	NOT NULL DEFAULT 0
is_primary	BOOLEAN	NOT NULL DEFAULT FALSE
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()

Design Notes

created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()

- one-to-many relationship
- one product - many images

Payments Table

Column	Type	Notes
id	INTEGER GENERATED ALWAYS AS IDENTITY	PRIMARY KEY
order_id	INTEGER	NOT NULL REFERENCES orders(id)
stripe_payment_intent_id	TEXT	UNIQUE
amount	DECIMAL(10,2)	NOT NULL CHECK (amount > 0)
status	VARCHAR(30)	NOT NULL CHECK (status IN ('succeeded','failed','refunded','pending'))
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()

Relationships

Relationship	Cardinality	Notes
User → Address	one-to-many	One user can have multiple saved addresses
User → Cart	one-to-zero-or-many	Nullable — guest carts exist without a user
User → Order	one-to-many	One user can place many orders
Address → Order (shipping)	one-to-many	Distinct from billing — needs its own line
Address → Order (billing)	one-to-many	Distinct from shipping — needs its own line
Category → Product	one-to-many	One category holds many products
Category → Category	one-to-many (self-referencing)	parent_category_id loops back into same table
Product → Product_Images	one-to-many	Multiple photos per product
Product → Product_Variant	one-to-many	Sizes/colors branch off one base product
Product_Variant → Inventory	one-to-one	Each SKU has exactly one stock record
Product_Variant → Cart_Items	one-to-many	Same variant can appear in many different carts
Product_Variant → Order_Items	one-to-many	Same variant can appear across many past orders
Cart → Cart_Items	one-to-many	One cart holds many line items
Order → Order_Items	one-to-many	One order holds many line items
Order → Payment	one-to-one	One payment transaction per order (scoped-down simplification)
Discount → Order	one-to-many	One code can be reused across many orders; each order applies at most one

Implementation Order

Step	Table	Why it can go here
1	users	No foreign keys — nothing to wait on
2	categories	Only references itself (parent_category_id), so no external wait
3	inventory	No foreign keys
4	discounts	No foreign keys
5	addresses	Needs users to exist (user_id)
6	products	Needs categories to exist (category_id)
7	product_variants	Needs products and inventory to exist
8	product_images	Needs products to exist
9	cart	Needs users to exist
10	orders	Needs users, addresses, and discounts to exist
11	cart_items	Needs cart and product_variants to exist
12	order_items	Needs orders and product_variants to exist
13	payments	Needs orders to exist